KG 구축 how 보완 (2)
Relation 잇기 · 저장 · 검색·답변

따른 문헌의 원리와 실제 수행 과정

2026.11 · 문경민

1. KG 구축 단계와 방법론 구체화 현황

- KG 구축은 두 단계로 한다. 먼저 온톨로지의 Class 와 Data Property 에 맞춰 자료에서 실제값으로 Entity 와 그 Data Property 를 만들고, 다음에 만든 Entity 들을 온톨로지의 Relation 에 맞춰 이어 KG 를 완성한다.
- 10/30 자료에서 앞 단계를 정리했고, 그 결과 Entity 264 개와 Data Property 값 616 개가 만들어졌다. 이번 자료는 Entity 를 Relation 으로 잇는 과정(2 장), 완성한 KG 를 Neo4j 에 넣는 저장(3 장), KG 로 질문에 답하는 검색·답변(4 장)을 같은 방식(따른 원리 → 과정 → 결과)으로 정리했다.
- 온톨로지의 CQ 와 Relation 은 부록 B, 10/30 자료까지의 결과는 부록 C, 각 원리의 자세한 설명은 부록 D 에 있다.

표 1. 단계별 하는 일과 방법론 정리 여부

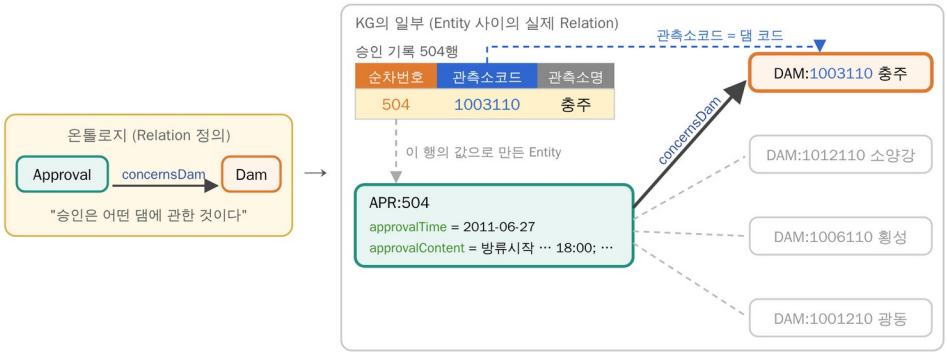
단계	하는 일	방법	방법론 정리
자료 준비	소스 구분과 전처리	—	정리함 (10/30)
Class·Data Property 채우기	Entity 와 속성값을 채움	구조화 자료 매핑(정형 표), 비정형 텍스트 추출(비고, 별표 3)	정리함 (10/30)
Relation 잇기	만든 Entity 를 Relation 마다 실제 시작·끝 Entity 로 이음	키로 이을 끝 Entity 를 찾음(댐 표기 맞추기 포함)	이번 (2 장)
저장	완성한 KG 를 Neo4j 에 넣음	CSV → Cypher → Neo4j	이번 (3 장)
검색·답변	질문에 맞는 근거를 KG 에서 찾아 LLM 이 답함	의도 추출 → Cypher 템플릿 조회 → 근거 묶음 → 답변	이번 (4 장)
검증	만든 KG 가 맞는지 확인	—	선행연구 조사 후

2. Relation 잇기

2.1 Relation 잇기란

- 그림 1 의 왼쪽은 온톨로지의 Relation 정의 "Approval —concernsDam→ Dam"이다. 승인 Class 와 댐 Class 사이에 이 Relation 이 있을 수 있다는 정의라서, 어느 승인이 어느 댐에 관한 것인지는 들어 있지 않다.
- 오른쪽은 KG 의 일부이다. 승인 APR:504 에서 댐 4 개로 가는 선은 모두 이 정의에 맞는다(회색 점선). 승인 80 개 × 댐 4 개 = 320 가지 조합이 정의에 맞지만, 실제 Relation 은 80 개뿐이다.
- 어느 댐과 이을지는 승인 기록 원본의 값이 알려 준다. APR:504 를 만든 승인 504 행의 관측소코드가 1003110 이고, 이 값은 충주댐의 댐 코드와 같다. 그래서 APR:504 를 DAM:1003110(충주댐)과 잇는다(실선).

- Entity 와 Data Property 값만 있는 KG 에는 선이 하나도 없어서 "충주댐의 승인"을 조회해도 아무것도 나오지 않는다. 이번 단계에서 Entity 를 Relation 으로 이어 KG 를 완성한다.



온톨로지의 Relation 정의만 보면 승인 80개 × 댐 4개 = 320가지 조합이 모두 맞다(회색 점선). 실제로 이을 댐은 승인 기록의 관측소코드 1003110과 같은 댐 코드를 가진 DAM:10031100이다(실선).

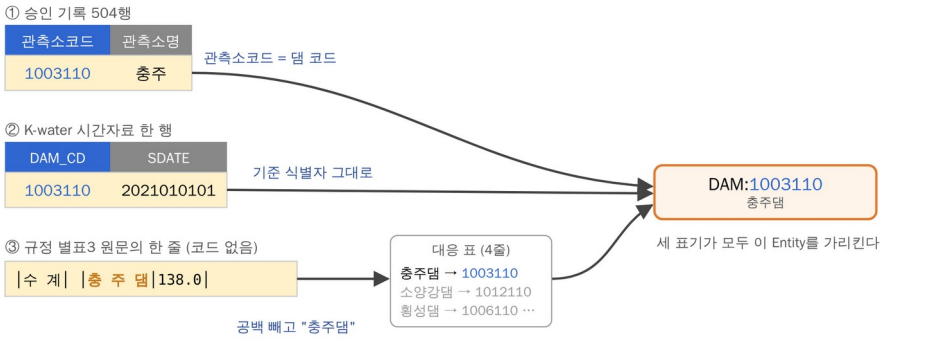
그림 1. 온톨로지의 Relation 정의에 맞는 조합(회색 점선) 가운데, 승인 기록의 관측소코드로 실제로 이을 댐(실선)을 찾는다

2.2 따른 원리와 진행 방식

- 따른 원리: Purohit 외(2024)는 GraphAide 에서 뉴스 기사 1,846 건으로 KG 를 만들 때, 글에서 뽑은 대상의 표기를 기준 지식베이스(DARPA Wikidata)의 식별자(QID)에 대응시켰다. 같은 QID 를 받은 표기는 같은 Entity 가 된다(부록 D.1).
- 원리: 기준 식별자를 하나 정하고, 자료마다 다르게 적힌 같은 대상을 그 식별자에 대응시킨다. 우리는 K-water 댐 코드를 기준 식별자로 삼았다. 이번에는 대상이 댐 4 개라서 코드 일치와 대응 표를 적용했다. GraphAide 의 벡터 검색과 LLM 재순위는 대상과 자료가 늘어날 때 적용을 검토한다.
- 참고로 Huang 외(2026)는 댐마다 관련 Entity 를 JSON 파일 하나로 묶어 적재했고, 본문에 서로 다른 자료의 같은 대상을 맞추는 단계는 따로 없다. 우리 자료는 승인 기록, K-water 자료, 규정이 서로 다른 기관에서 따로 와서 댐 표기를 맞추는 과정이 필요하다.
- 진행 방식: 먼저 자료마다 다르게 적힌 댐 표기를 기준 식별자(댐 코드)에 맞춘다. 그 다음 Relation 마다 시작 Entity 를 하나씩 읽고, 키로 이을 끝 Entity 를 찾아 Relation 한 줄을 쓴다.
- 키는 두 Entity 가 같은 대상에 관한 것임을 알려 주는 값이다(승인 순차번호, 댐 코드, 댐 이름). 시작 Entity 와 끝 Entity 는 Relation 한 줄의 출발점과 도착점이다. 예를 들어 "APR:504 —concernsDam→ DAM:1003110"에서 APR:504 가 시작 Entity, DAM:1003110 이 끝 Entity 이다.

2.3 댐 표기 맞추기 (개체 해소)

- 그림 2 의 왼쪽은 충주댐이 자료마다 적힌 모습이다. ① 승인 기록은 관측소코드 1003110 과 관측소명 "충주", ② K-water 자료는 DAM_CD 1003110, ③ 규정 별표 3 은 "충 주 댐"이라는 이름뿐이고 코드가 없다.
- ① 은 관측소코드가 K-water 댐 코드와 같아서 그대로 맞춘다(자료 준비 때 4 개 댐 모두 같음을 확인).
- ② 는 기준 식별자 자체이다.
- ③ 은 비정형 텍스트 추출에서 공백을 빼고 읽은 "충주댐"을 대응 표(표준 댐명 → 댐 코드, 4 줄)에서 찾아 1003110 을 얻는다. 광동댐은 별표 3 에 없다.
- 결과: 세 표기가 모두 하나의 Entity DAM:1003110 을 가리킨다.

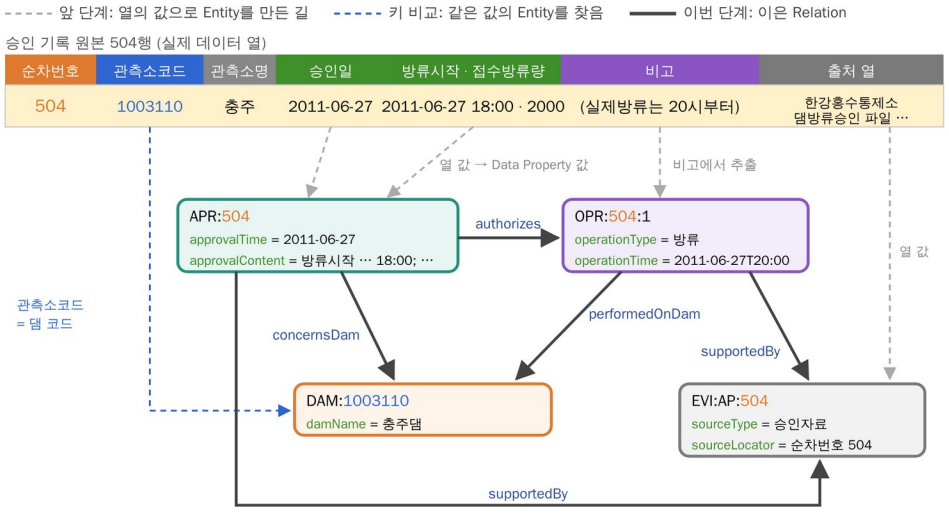


원리 · Purohit 외(2024) — 기준 식별자(QID)를 정하고, 자료마다 다르게 적힌 같은 대상을 그 식별자에 대응시킨다. 우리의 기준 식별자는 K-water 댐 코드이다.

그림 2. 자료마다 다른 충주댐 표기 세 가지를 기준 식별자 DAM:1003110 에 맞춘다

2.4 승인 기록 한 행으로 잇는 Relation

- 승인 기록 한 행에서 키를 얻어 잇는 Relation 은 다섯 가지이다. 그림 3 의 위쪽은 승인 기록 원본 파일(CSV)의 504 행을 실제 데이터 열 그대로 그린 것이다.
- 그림 위의 범례처럼 회색 점선은 앞 단계에서 열의 값으로 Entity 를 만든 길, 파란 점선은 키가 같은 Entity 를 찾는 길, 검은 실선은 이번 단계에서 이은 Relation 이다.
- 앞 단계에서 승인일·방류시작·접수방류량 열로 APR:504 를, 비고를 추출해 OPR:504:1 을, 출처 열로 EVI:AP:504 를 만들었다(회색 점선). 세 ID 에 모두 이 행의 순차번호 504(주황)가 들어 있다.
- 실선이 이번에 잇는 Relation 이다. ① APR:504 → OPR:504:1(authorizes), ② APR:504 → EVI:AP:504 와 OPR:504:1 → EVI:AP:504(supportedBy)는 ID 의 순차번호가 같은 것끼리 잇는다. ③ APR:504 → DAM:1003110(concernsDam)과 OPR:504:1 → DAM:1003110(performedOnDam)은 이 행의 관측소코드 1003110 과 같은 댐 코드를 가진 DAM:1003110 을 끝 Entity 로 찾는다(파란 점선).
- 결과: 승인 80 행에 같은 방법을 적용해 supportedBy 148 개(승인 80, 운영행위 68), authorizes 68 개, concernsDam 80 개, performedOnDam 68 개를 잇는다. 변경 승인 17 개는 운영행위가 없어 authorizes 가 없다.



점선은 열의 값으로 Entity와 그 Data Property 값을 만든 길이고, 실선이 Relation이다. 한 행에서 만든 Entity 3개(APR, OPR, EVI:AP)의 ID에 모두 순차번호 504가 들어 있어, 같은 순차번호끼리 잇는다(authorizes, supportedBy). 댐은 이 행의 관측소코드 1003110과 같은 댐 코드를 가진 DAM:1003110을 찾아 잇는다(concernsDam, performedOnDam).

그림 3. 승인 504 행 하나에서 잇는 Relation — 순차번호 504 와 관측소코드 1003110 으로 이을 Entity 를 찾는다

2.5 댐 코드로 잇는 Relation

- 그림 4 의 위쪽은 K-water 시간자료의 충주댐 한 행이다. 앞 단계에서 이 표의 DAM_CD 와 열 머리(변수)로 HMS 6 개와 측정자료 근거 EVI:KW 하나를 만들었다(회색 점선). 댐 Entity 는 일자료에서 만들었고, 이 행의 DAM_CD 1003110 과 같은 댐 코드로 찾는다(파란 점선).
- 세 종류 Entity 의 ID 에 모두 같은 댐 코드 1003110(파랑)이 들어 있다. 그래서 ① DAM:1003110 → HMS 6 개 (hasHydrometeorologicalState), ② HMS 6 개 → EVI:KW:1003110(supportedBy)을 코드가 같은 것끼리 잇는다. 다른 자료를 찾을 필요가 없다.
- 결과: 4 개 댐 × 6 개 변수로 hasHydrometeorologicalState 24 개, supportedBy 24 개를 잇는다.

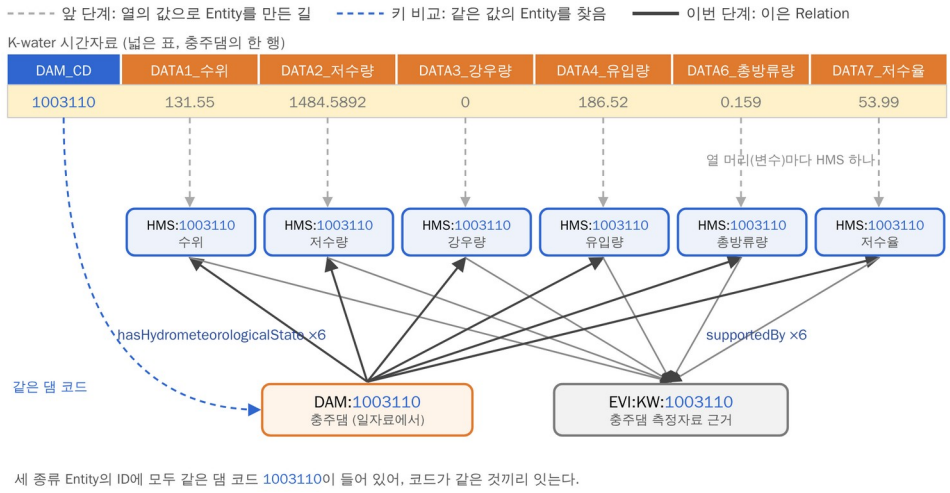


그림 4. 댐 코드로 잇는 Relation — 같은 코드 1003110 을 가진 Dam, HMS, 측정자료 근거를 잇는다

2.6 별표 3 댐 이름으로 잇는 Relation

- 그림 5 의 위쪽은 비정형 텍스트 추출로 만든 중간 파일 criterion_list.csv 의 한 줄이다. 앞 단계에서 이 줄로 CRI:CR-02 를 만들었다(회색 점선).
- 별표 3 에는 댐 코드가 없다. 그래서 ① 이 줄의 댐 이름 "충주댐"을 대응 표에서 찾아 댐 코드 1003110 을 얻고(파란 점선), ② CRI:CR-02 → DAM:1003110(appliesToDam)을 쓴다. 코드가 아니라 이름으로 찾은 끝 Entity 라서, Relation 한 줄에 이름으로 찾았다는 표시(name-match)를 남긴다.
- ③ 근거는 규정 하나뿐이라 근거 Entity 도 하나(고정 ID EVI:LAW:별표 3)이다. 기준 3 개 모두 CRI → EVI:LAW:별표 3(supportedBy)으로 잇는다.
- 결과: appliesToDam 3 개(충주, 소양강, 횡성), supportedBy 3 개를 잇는다.

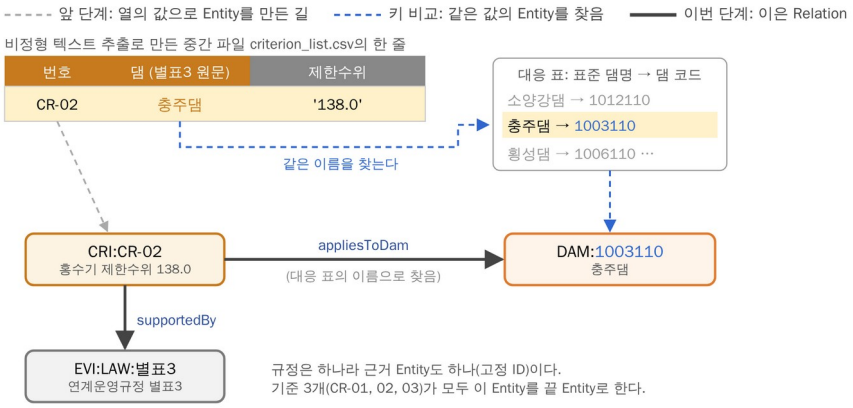


그림 5. 별표 3 댐 이름으로 잇는 Relation — 이름을 대응 표로 댐 코드에 맞춘다

2.7 결과: Relation 을 이은 CSV

- 온톨로지의 Relation 9 종에 대해 실제 Relation 418 개가 kgv4_relations.csv 에 한 줄씩 쓰였다. 그림 6 은 그 파일의 실제 줄이다.

- 한 줄은 Relation 하나이며, 시작 Entity(subject), Relation 이름(relation), 끝 Entity(object), 끝 Entity 를 찾은 방법의 이름(rule), 원본의 키로 바로 이었는지 표시(origin), 설명(note)으로 이루어진다. 예를 들어 "APR:504, concernsDam, DAM:1003110"은 승인 504 가 충주댐에 관한 것이라는 Relation 하나이다.
- 이로써 KG 에 들어갈 내용이 CSV 파일 세 개에 모두 담겼다. kgv4_entities(Entity 와 Class), kgv4_properties(Data Property 값), kgv4_relations(Relation)이다.
- 연결 확인: 4 개 댐 전체에서 승인 80 개 모두 댐과 근거에, 운영행위 68 개 모두 댐·승인·근거에, HMS 24 개 모두 댐과 근거에, 기준 3 개 모두 댐과 근거에 이어졌다. 승인 중 운영행위로 이어진 것은 63 개이며, 나머지 17 개는 변경 승인이다.

kgv4_relations.csv — 전체 418줄 중 충주댐의 Entity 몇 개(승인 504, 운영행위 504:1, 수위 HMS, 기준 CR-02, 댐)가 시작인 줄 14 개

subject (시작 Entity)	relation	object (끝 Entity)	rule	origin	note
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:수위	MR-HMS	real	
HMS:1003110:수위	supportedBy	EVI:KW:1003110	MR-SUP-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:저수량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:강우량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:유입량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:총방류량	MR-HMS	real	
DAM:1003110	hasHydrometeorologicalState	HMS:1003110:저수율	MR-HMS	real	
APR:504	supportedBy	EVI:AP:504	MR-SUP-APR	real	
APR:504	concernsDam	DAM:1003110	MR-APR-DAM	real	승인 레코드의 관측소코드 = 댐 코드 (DAM_CD)
OPR:504:1	performedOnDam	DAM:1003110	MR-OPR-DAM	derived	승인 행의 시설 코드
APR:504	authorizes	OPR:504:1	MR-AUTH	constructed	같은 승인 행의 비고에서 분류한 행위 (연구용 구성, 실행 확인 아님)
OPR:504:1	supportedBy	EVI:AP:504	MR-SUP-OPR	derived	
CRI:CR-02	appliesToDam	DAM:1003110	MR-CRI-DAM	name-match	별표3 시설명 일치(코드 없음)
CRI:CR-02	supportedBy	EVI:LAW:별표3	MR-SUP-CRI	real	
… 나머지 404줄					

그림 6. kgv4_relations.csv 의 실제 줄 — 충주댐 Entity 몇 개가 시작인 줄 14 개

3. 저장

3.1 따른 원리와 진행 방식

- 따른 원리: Huang 외(2026)는 추출한 지식을 JSON 으로 모은 뒤, 온톨로지의 관계에 따라 Python 스크립트가 JSON 의 키-값을 읽어 Cypher 문을 만들고, Entity 와 Relation 을 Neo4j 에 한꺼번에 넣었다. Neo4j 는 Cypher 로 조회·추가·삭제·수정이 효율적이고 여러 Relation 을 건너는 질의에 강해서 골랐다고 밝혔다(부록 D.2).

- 따른 원리 (KG 밖에 두는 저장소): Zhou 외(2023)는 댐 안전 모니터링 데이터를 MySQL 에 그대로 둔 채, 표의 키 열 값으로 온톨로지 인스턴스의 식별자를 만들어 행과 인스턴스를 대응시켰다(D2RQ 매핑). 질의가 들어오면 이를 SQL 로 바꿔 MySQL 에서 그 행을 읽어 오며, 데이터를 그래프 저장소로 복제하지 않는다(부록 D.3).
- Neo4j 는 Entity 와 Relation 을 그래프(점과 선) 모양으로 저장하는 데이터베이스이다. Cypher 는 Neo4j 에 Entity 와 Relation 을 넣고 찾을 때 쓰는 명령 문장이다. CSV 는 줄의 목록이라 "충주댐에서 출발해 관계를 따라간다"는 질의를 바로 할 수 없어서, 그래프로 저장한다.
- 우리는 모은 형태가 JSON 이 아니라 2 장까지 만든 CSV 파일 세 개이다. Python 스크립트가 이 파일의 줄을 읽어 Cypher 문을 쓰고, 그 문을 모은 파일을 Neo4j 에서 실행한다. 출처 파일(kgv4_provenance)은 Neo4j 에 넣지 않고 CSV 로 둔다. 시간별 측정값과 규정 원문도 Zhou 외(2023)의 원리에 따라 KG 밖 CSV 에 두고, KG 에는 그 행을 찾을 키만 둔다.

3.2 CSV → Cypher → Neo4j

과정 1. Entity 를 Cypher 문으로 쓰고 Neo4j 에 넣기

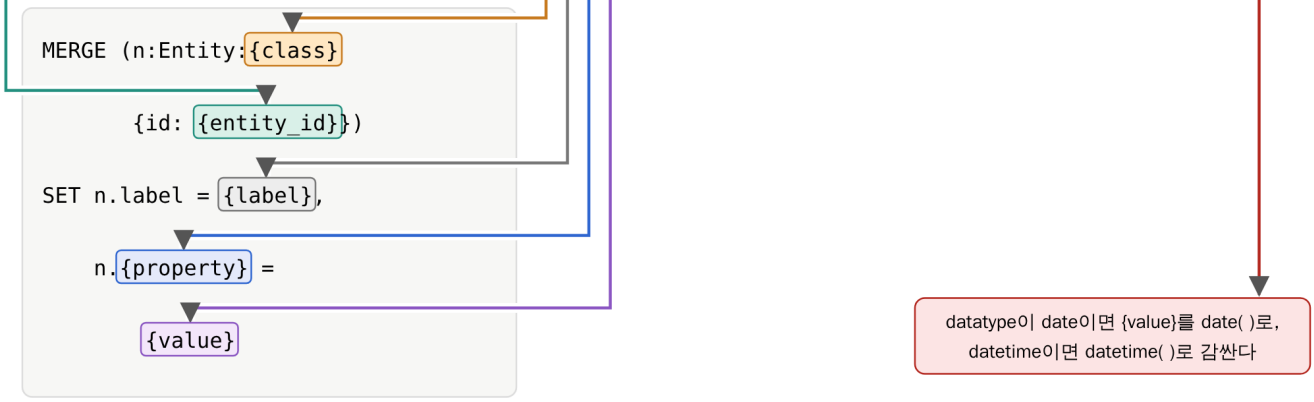
- 원리: Entity 하나는 Cypher 문 하나가 되고, 실행하면 Neo4j 에 Entity 하나가 생긴다. Class 와 Data Property 값도 그 문 안에 함께 들어간다.
- 그림 7 의 맨 위는 입력이다. 왼쪽은 kgv4_entities.csv 의 APR:504 줄, 오른쪽은 kgv4_properties.csv 의 APR:504 줄 2 개이며, Python 스크립트가 이 줄들을 Entity ID 로 모은다.
- 가운데는 Cypher 문의 틀이다. 화살표를 따라가면 entity_id 열의 값은 {entity_id} 자리에, class 열은 :Entity 뒤의 {class} 자리에, label 열은 n.label = 뒤에 들어간다. properties 의 줄마다 property 열은 n. 뒤의 {property}에, value 열은 = 뒤의 {value}에 들어가 한 줄씩 생긴다. datatype 열이 date 이면 값을 date( )로 감싸 글자가 아니라 날짜로 넣는다. 그래야 "2020 년 8 월 안"처럼 기간으로 비교해 찾을 수 있다.
- 그 아래가 APR:504 의 값을 넣어 만든 실제 Cypher 문이고, 칠한 부분이 CSV 열에서 온 값이다. MERGE 는 이 Entity 가 없으면 만들고 있으면 그대로 쓰라는 뜻이라서, 같은 문을 다시 실행해도 APR:504 가 둘이 되지 않는다. SET 은 Data Property 값을 넣으라는 뜻이다.
- 맨 아래가 실행 결과이다. Neo4j 에 Entity APR:504 가 Class 와 Data Property 값을 가진 채 생긴다. 264 개 Entity 마다 이런 문이 하나씩 쓰인다.

kgv4_entities.csv (APR:504 줄)

entity_id	class	label
APR:504	Approval	승인 504 (충주댐)

kgv4_properties.csv (APR:504 줄 2개)

entity_id	property	value	datatype
APR:504	approvalTime	2011-06-27	date
APR:504	approvalContent	방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터)	string



{ } 자리에 CSV 열의 값이 들어간다. 맨 아래 두 줄(n.{property} = {value})은 properties의 줄마다 생긴다.

↓ APR:504의 줄 3개를 넣으면 (실제 파일의 한 줄)

```
MERGE (n:Entity:Approval {id: 'APR:504'})
SET n.label = '승인 504 (충주댐)',
    n.approvalTime = date('2011-06-27'),
    n.approvalContent = '방류시작 2011-06-27 18:00; …';
```

MERGE — 이 Entity가 없으면 만들고 있으면 그대로 쓴다. ( ) 하나가 Entity 하나이고 n은 이 문 안에서 부르는 이름이다.
:Entity — 모든 Entity에 붙는 공통 표시로, Class를 몰라도 ID 하나로 찾게 한다. SET — Data Property 값을 넣는다.

↓ Neo4j에서 실행하면

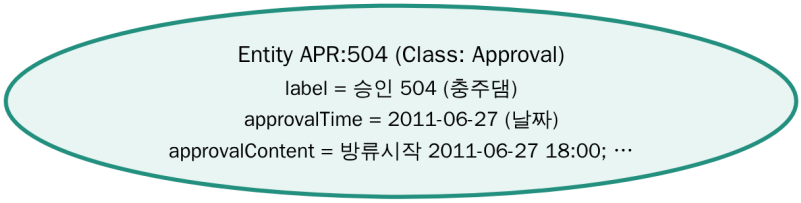
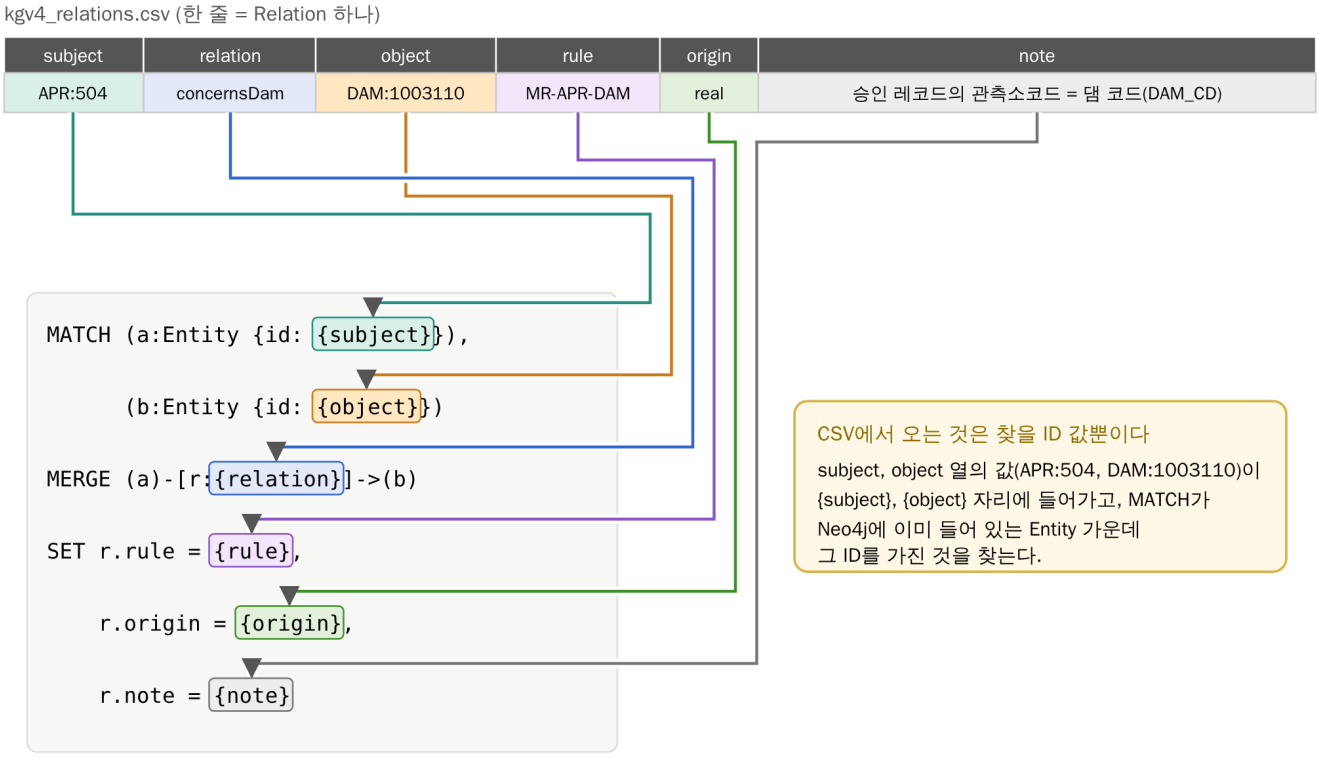


그림 7. 과정 1 — CSV 열의 값이 Cypher 문의 자리로 들어가고, 실행하면 Neo4j 에 Entity 가 생긴다 (APR:504)

과정 2. Relation 을 Cypher 문으로 쓰고 Neo4j 에 넣기

- 원리: Relation 하나는 Cypher 문 하나가 된다. Relation 은 두 Entity 사이에 긋는 것이므로, 문은 두 Entity 를 먼저 찾고 그 사이에 Relation 을 만든다.
- 그림 8 의 맨 위는 kgv4_relations.csv 의 한 줄이다. 가운데 틀에서 화살표를 따라가면 subject 와 object 열의 값은 MATCH 문에서 찾을 두 Entity 의 ID 자리({subject}, {object})에, relation 열은 화살표 모양 안의 {relation} 에, rule, origin, note 열은 SET 뒤의 값 자리에 들어간다. 그 아래가 이 줄의 값을 넣은 실제 Cypher 문이다.

- MATCH 는 Neo4j 에 이미 들어 있는 Entity 가운데 ID 가 APR:504 와 DAM:1003110 인 것을 찾으라는 뜻이고, 찾은 두 Entity 를 이 문 안에서 a 와 b 로 부른다. CSV 에서 가져오는 것은 찾을 ID 뿐이며, Entity 자체는 Entity 문으로 Neo4j 에 먼저 넣어 둔 것이다. Relation 은 Neo4j 안의 두 Entity 사이에 긋는 것이라 Entity 가 먼저 Neo4j 에 있어야 한다.
- (a)-[r:concernsDam]→(b)는 a 에서 b 로 가는 concernsDam Relation 이다. MERGE 는 a 와 b 사이에 이 Relation 이 이미 있으면 새로 만들지 않고 없을 때만 만든다. 그래서 같은 문을 두 번 실행해도 같은 선이 두 개 생기지 않는다.
- 맨 아래가 실행 결과이다. 두 Entity 사이에 방향이 있는 선 하나가 생기고, 그 선에 rule, origin, note 값이 남는다. 418 개 Relation 마다 이런 문이 하나씩 쓰인다.



↓ 이 줄을 넣으면 (실제 파일의 한 줄)

```
MATCH (a:Entity {id: 'APR:504'}), (b:Entity {id: 'DAM:1003110'})
MERGE (a)-[r:concernsDam]->(b)
SET r.rule = 'MR-APR-DAM', r.origin = 'real',
    r.note = '승인 레코드의 관측소코드 = 댐 코드(DAM_CD)';
```

MATCH — Neo4j에 이미 들어 있는 Entity 가운데 ID가 APR:504, DAM:1003110인 것을 찾아 a, b로 부른다.
MERGE (a)-[r:concernsDam]->(b) — a에서 b로 가는 Relation r이 없을 때만 만든다.
SET — Relation r에 rule, origin, note 열의 값을 넣는다.

↓ Neo4j에서 실행하면

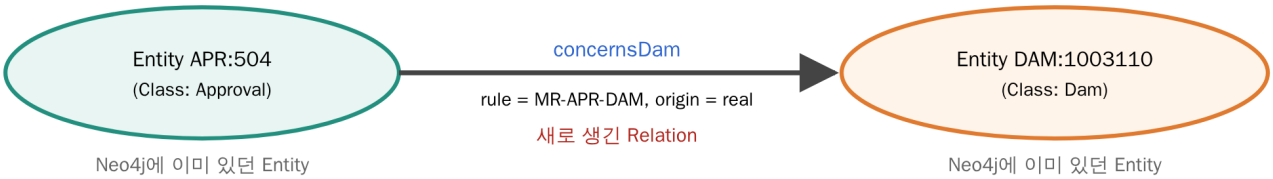


그림 8. 과정 2 — 관계 CSV 한 줄이 Cypher 문이 되고, 실행하면 두 Entity 사이에 Relation 이 생긴다

과정 3. Cypher 문을 모아 Neo4j 에서 실행하기

- 원리: 과정 1·2 에서 쓴 문을 파일 하나에 모아, Neo4j 가 위에서부터 한 번에 실행하게 한다.
- 그림 9 의 왼쪽이 그 파일(kgv4_load.cypher, 684 줄)이다. 주석 1 줄, ID 가 겹치지 않게 막는 제약 1 줄, Entity 문 264 줄, Relation 문 418 줄 순서이다. Relation 문은 두 Entity 를 찾아야 하므로 Entity 문이 모두 끝난 뒤에 온다.
- 오른쪽은 위에서 아래로 실행 순서이다. Neo4j 데이터베이스 서버를 시작하고, 이전에 넣은 Entity 와 Relation 을 모두 지운 뒤, 파일을 실행하고, 마지막에 Entity 와 Relation 수를 세어 264 개, 418 개인지 확인한다.
- 먼저 모두 지우는 이유: KG 의 내용은 CSV 에서 나오므로 CSV 를 고치면 Neo4j 도 고친 CSV 와 같아야 한다. 그런데 MERGE 는 없는 것을 더할 뿐, CSV 에서 지운 줄을 Neo4j 에서 지우지는 않는다. 예를 들어 잘못 이은 Relation 한 줄을 CSV 에서 지우고 파일만 다시 실행하면, 그 Relation 은 Neo4j 에 그대로 남는다. 그래서 매번 Neo4j 를 비운 뒤 파일 전체를 다시 실행한다. 마지막에 센 수가 CSV 의 줄 수(264, 418)와 같으면 Neo4j 의 KG 가 CSV 와 같다.

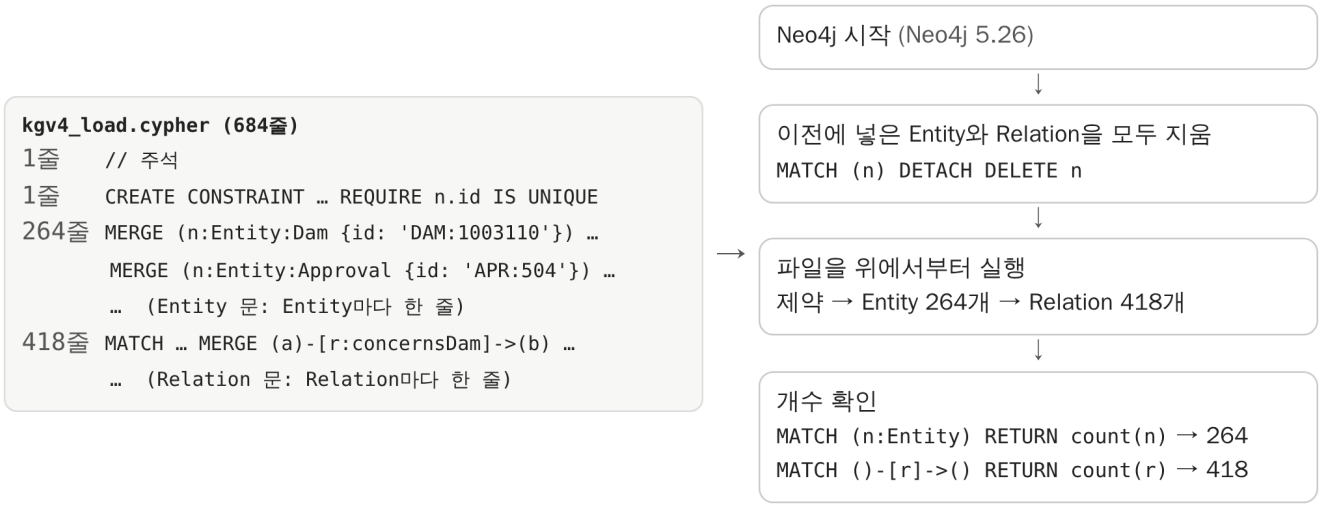


그림 9. 과정 3 — Cypher 파일의 구성과 실행 순서

3.3 KG 밖에 두는 것: 측정 저장소와 문서 저장소

- 시간별 측정값(508,246 행)과 규정 원문은 KG 에 넣지 않고 CSV 파일로 둔다. KG 에는 그 파일의 행을 찾아갈 키만 Data Property 로 두고, 조회할 때 그 키로 행을 읽는다. Zhou 외(2023)는 이를 매핑 파일로 구현했고, 우리는 키(stationCode 와 시각, sourceLocator)로 CSV 의 행을 찾는 코드로 구현했다.
- 그림 10 의 위는 측정 저장소이다. Entity HMS:1003110:수위의 stationCode(1003110)가 DAM_CD 열과 같고, 찾을 시각(2020-08-02 18:00)이 SDATE_raw 열(2020080218)과 같은 행을 찾는다. 그 행의 DATA1_수위 열이 그 시각의 수위 139.55 EL.m 이다.
- 아래는 문서 저장소이다. Entity EVI:LAW:별표 3 의 sourceLocator("별표 3")가 sourceLocator 열과 같은 행을 찾으면, 그 행의 text 가 규정 원문이다.

측정 저장소 — KG의 Entity가 가진 값(stationCode, 시각)으로 CSV의 행을 찾는다

Entity HMS:1003110:수위
stationCode = 1003110
variableType = 수위

→ DAM_CD = stationCode, SDATE_raw = 찾는 시각(2020-08-02 18:00 → 2020080218), 열 = variableType이 든 열(DATA1_수위)

kwater_mywater_hydr_H_aligned_wide.csv.gz (2010-08~2020-12) · kwater_mywater_hydr_H_wide.csv.gz (2021-01~2026-09)도 같은 열 · 4개 댐 합계 508,246행

source_file	retrieval_time_utc	param1_resolution	DAM_CD	SDATE_raw	DATA1_수위(EL.m)	DATA2_저수량(MCM)	DATA3_강우량(mm)	DATA4_유입량(CMS)	DATA5_source_label_unknown	DATA6_총방류량(CMS)	DATA7_저수율(%)
01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json	2026-10-09T01:36:21Z	H	1003110	2020080217	139.37	2111.215	0.711	4693.654	4693.654	707.265	76.8
01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json	2026-10-09T01:36:21Z	H	1003110	2020080218	139.55	2126.472	1.901	4945.201	4945.201	707.145	77.3
01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json	2026-10-09T01:36:21Z	H	1003110	2020080219	139.69	2138.381	0.273	4015.509	4015.509	707.453	77.8

문서 저장소 — KG의 Entity가 가진 값(sourceLocator)으로 CSV의 행을 찾는다

Entity EVI:LAW:별표3
sourceLocator = 별표 3

→ sourceLocator 열이 "별표 3"인 행의 text가 규정 원문이다

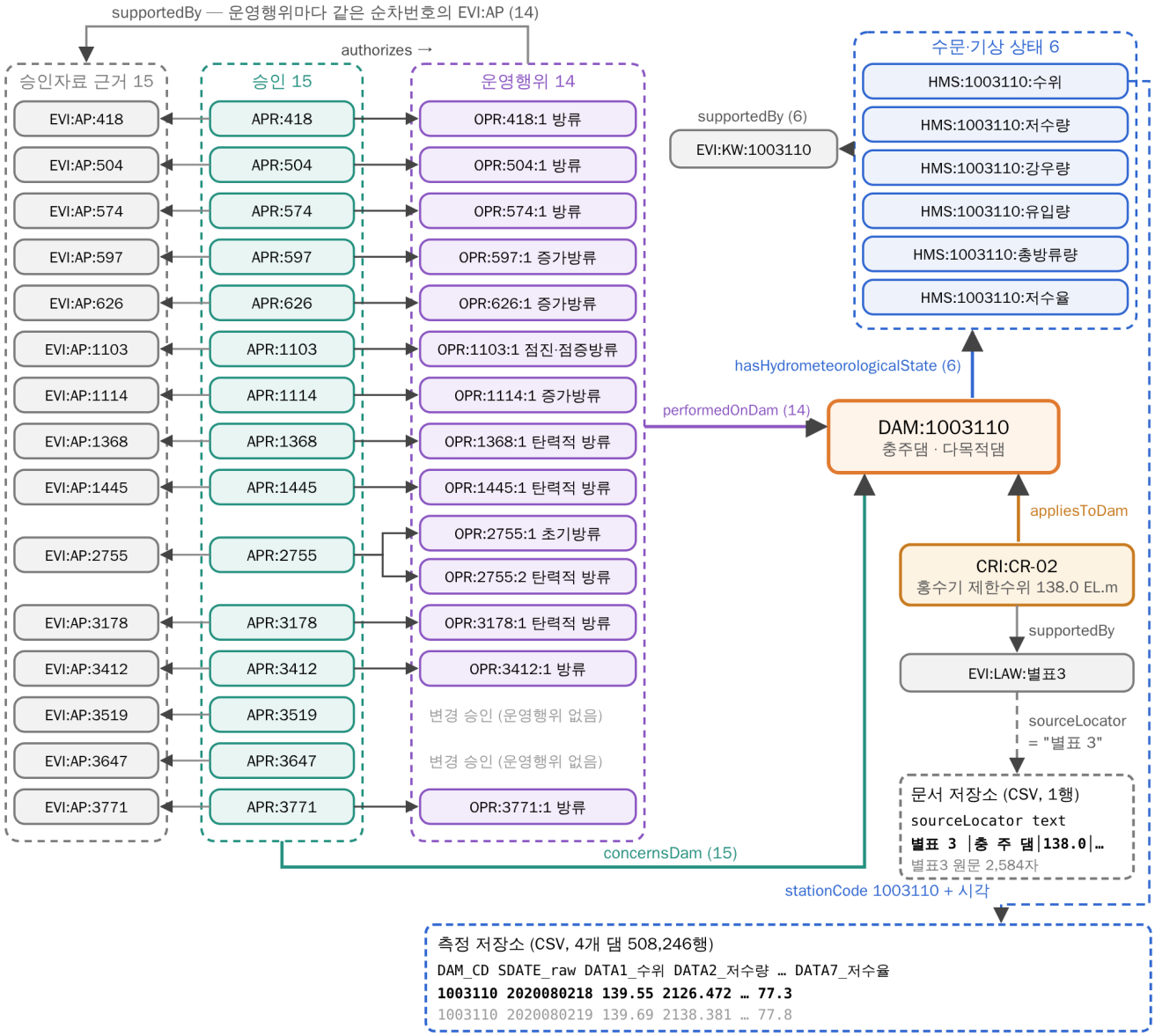
kgv4_documents.csv (1행)

sourceLocator	doc_title	text (2,584자 중 앞부분)
별표 3	댐과 보 등의 연계운영규정 (기후에너지환경부 훈령 제42호, 시행 2026-07-08)	댐의 홍수기 제한수위(제6조) [별표 3] 댐의 홍수기 제한수위(제6조) ┌──┬──┬──┬──┬──┐ |구 분 |댐 종류 |댐 명 |홍수기 제한수위 |시설별 최저 운영수위| | | | | (EL.m) | (EL.m) | ├──┼──┼──┼──┼──┤ |한 강 |다목적댐 |소양강댐 |190.3 |- |수 계 | |충 주 댐 |138.0 |- | | |횡 성 댐 |178.2 |- | ├──┼──┼──┤ | |수력발전댐|화 천 댐 |175.0 |- | | |춘 천 댐 |102.0 |98.0 | | |의 암 댐 | 70.5 |67.0 …

그림 10. 외부 저장소 CSV 의 실제 모습과, KG 의 값으로 행을 찾는 방법

3.4 결과: Neo4j 에 저장된 KG

- Neo4j 에 Entity 264 개와 Relation 418 개가 저장된다(표 2).
- 그림 11 은 Neo4j 에 저장된 충주댐의 Entity 53 개와 Relation 이다. 왼쪽 세 점선 상자는 승인자료 근거 15, 승인 15, 운영행위 14 개이고, 한 줄이 승인 한 건이다. 승인마다 왼쪽 근거(supportedBy)와 오른쪽 운영행위(authorizes)로 이어지며, 3519 와 3647 은 변경 승인이라 운영행위가 없다.
- 상자에서 나가는 굵은 선은 상자 안의 Entity 가 모두 가진 Relation 이다. 승인 15 개는 모두 DAM:1003110 에(concernsDam), 운영행위 14 개는 모두 DAM:1003110 에(performedOnDam)와 같은 순차번호의 근거에(supportedBy) 이어진다. 오른쪽 위는 DAM:1003110 의 HMS 6 개와 그 근거, 오른쪽 아래는 기준 CR-02 와 그 근거이다.
- 맨 아래 두 점선 상자는 KG 에 넣지 않은 외부 저장소이고, 점선 화살표는 Relation 이 아니라 KG 의 값으로 그 행을 찾아오는 길이다. HMS 의 stationCode 와 시각으로 측정값을, EVI:LAW:별표 3 의 sourceLocator 로 규정 원문을 가져온다.



실선은 Neo4j에 저장된 Relation이고, 점선 상자(승인, 운영행위, 근거, HMS) 안의 Entity는 각각 그 Relation을 가진다(괄호 안은 개수). 맨 아래의 점선 화살표는 Relation이 아니라, KG의 값으로 외부 저장소의 행을 찾아오는 길이다.

그림 11. Neo4j 에 저장된 KG(충주댐 Entity 53 개)와 외부 저장소로 가는 길

표 2. Neo4j 에 저장된 Entity 와 Relation 수

Class	Entity 수	Relation	Relation 수
EvidenceSource	85	supportedBy	175
Approval	80	concernsDam	80
Operation	68	authorizes	68
HydrometeorologicalState	24	performedOnDam	68
Dam	4	hasHydrometeorologicalState	24
Criterion	3	appliesToDam	3
합계	264	합계	418

4. 검색·답변

4.1 따른 원리와 진행 방식

- 따른 원리: Purohit 외(2024)의 GraphAide 는 질의 단계에서 LLM 이 자연어 질문을 부분 그래프 조회 템플릿으로 바꾸고, 이를 Cypher 로 Neo4j 에서 실행한 결과를 LLM 에 맥락으로 주어 답변을 만든다. KG 의 관계를 따라 찾은 근거를 주므로 LLM 이 없는 사실을 지어내는 것(환각)을 줄인다(부록 D.1).
- GraphAide 는 이 밖에 질의 확장, Cypher 자동 생성, 벡터 검색, 에이전트를 함께 쓴다. 이번에는 구조를 시험하기 위해 KG 조회 결과를 LLM 의 근거로 주는 원리를 적용하고, 조회 템플릿은 CQ 마다 미리 정해 두었다. 나머지 방법은 답할 수 있는 질문의 범위를 넓힐 때 적용을 검토한다.
- 진행 방식: 질문 하나가 의도·값 추출(LLM) → 입력 확인(코드) → KG 검색과 외부 저장소 조회(코드) → 근거 묶음(코드) → 답변(LLM)의 다섯 과정을 거친다. LLM 은 처음과 마지막 두 곳에서 쓰고(같은 모델 Qwen3-32B-AWQ, 지시문만 다름), 근거를 찾는 가운데 세 과정은 코드가 한다.
- 아래에서는 서버에서 실행한 질문 하나(Qwen3-32B-AWQ, 2026-10-16)를 따라간다.

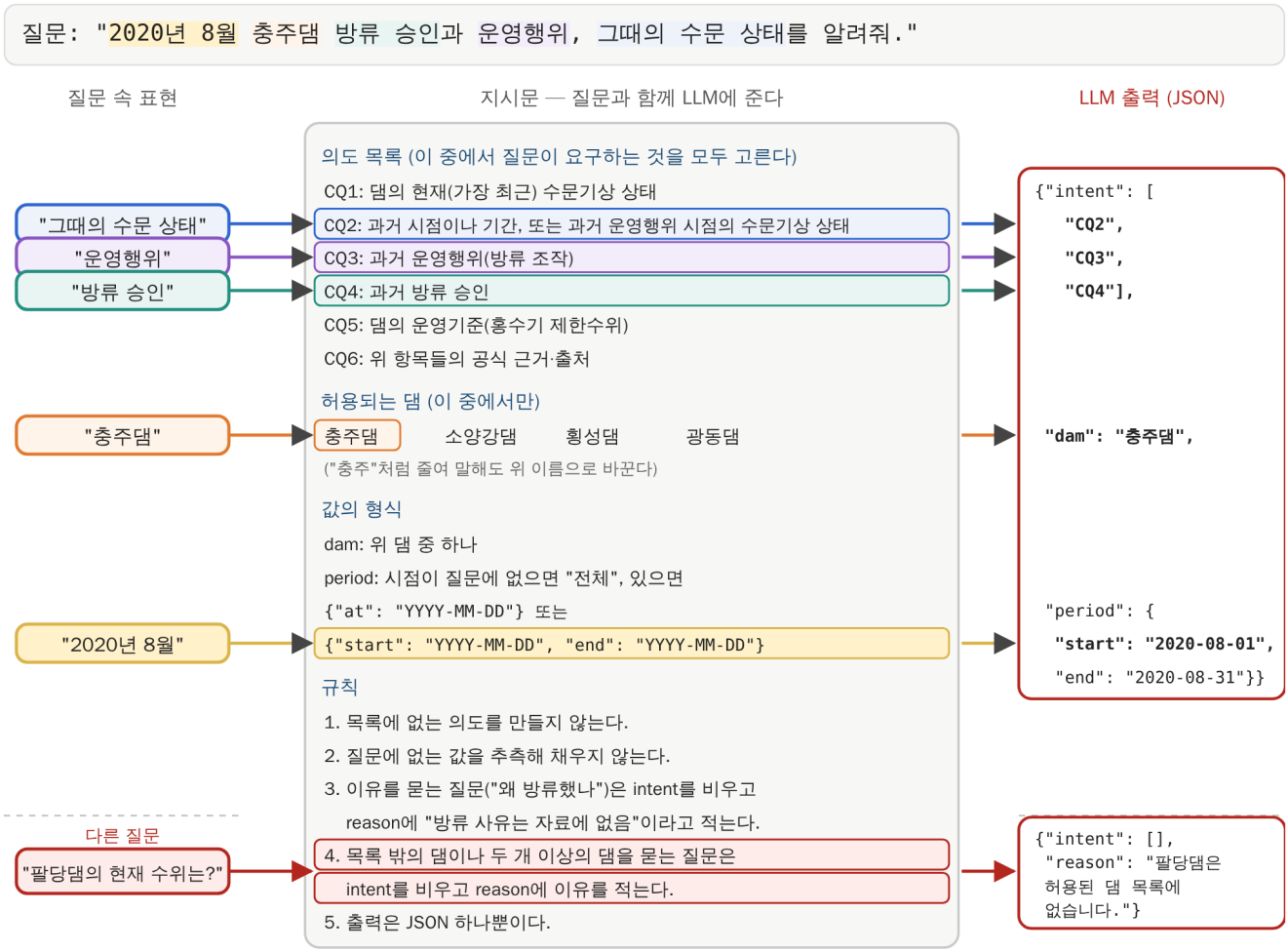
4.2 질문에서 근거 묶음까지

입력

- 질문: "2020 년 8 월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘."

과정 1. 의도·값 추출 (LLM)

- 원리: LLM 은 질문을 자유롭게 해석하지 않고, 지시문에 정해 둔 의도 목록 안에서 고른다. 그래서 LLM 에 질문과 함께 지시문을 준다.
- 그림 12 의 가운데가 그 지시문이다. 의도 목록은 온톨로지의 CQ1~CQ6 이며, 질문이 요구하는 것을 모두 고르게 한다. 허용되는 댐은 KG 에 있는 4 개 댐이다. 값의 형식은 Cypher 템플릿에 그대로 넣을 수 있는 모양으로 정해 두었다. 규칙 5 개는 목록 밖의 의도나 질문에 없는 값을 만들지 않게 한다.
- 왼쪽은 질문 속 표현, 오른쪽은 LLM 이 낸 JSON 이다. 화살표를 따라가면 "그때의 수문 상태"는 CQ2, "운영행위"는 CQ3, "방류 승인"은 CQ4 에 맞아 intent 가 된다. "충주댐"은 허용되는 댐에 있어 dam 이 되고, "2020 년 8 월"은 기간 형식에 맞춰 2020-08-01 ~ 2020-08-31 이 된다. CQ1, CQ5, CQ6 에 맞는 표현은 없어 고르지 않았다.
- 아래의 다른 질문 "팔당댐의 현재 수위는?"에서는 팔당댐이 허용되는 댐에 없으므로, LLM 이 규칙 4 에 따라 intent 를 비우고 이유를 적었다(같은 서버 실행). 이때는 KG 를 조회하지 않는다.



LLM 출력 원문: `{"intent": ["CQ2", "CQ3", "CQ4"], "dam": "충주댐", "period": {"start": "2020-08-01", "end": "2020-08-31"}}`

그림 12. 의도·값 추출 — 질문 속 표현을 지시문의 의도 목록, 허용되는 댐, 값의 형식에 맞춰 JSON 으로 적는다

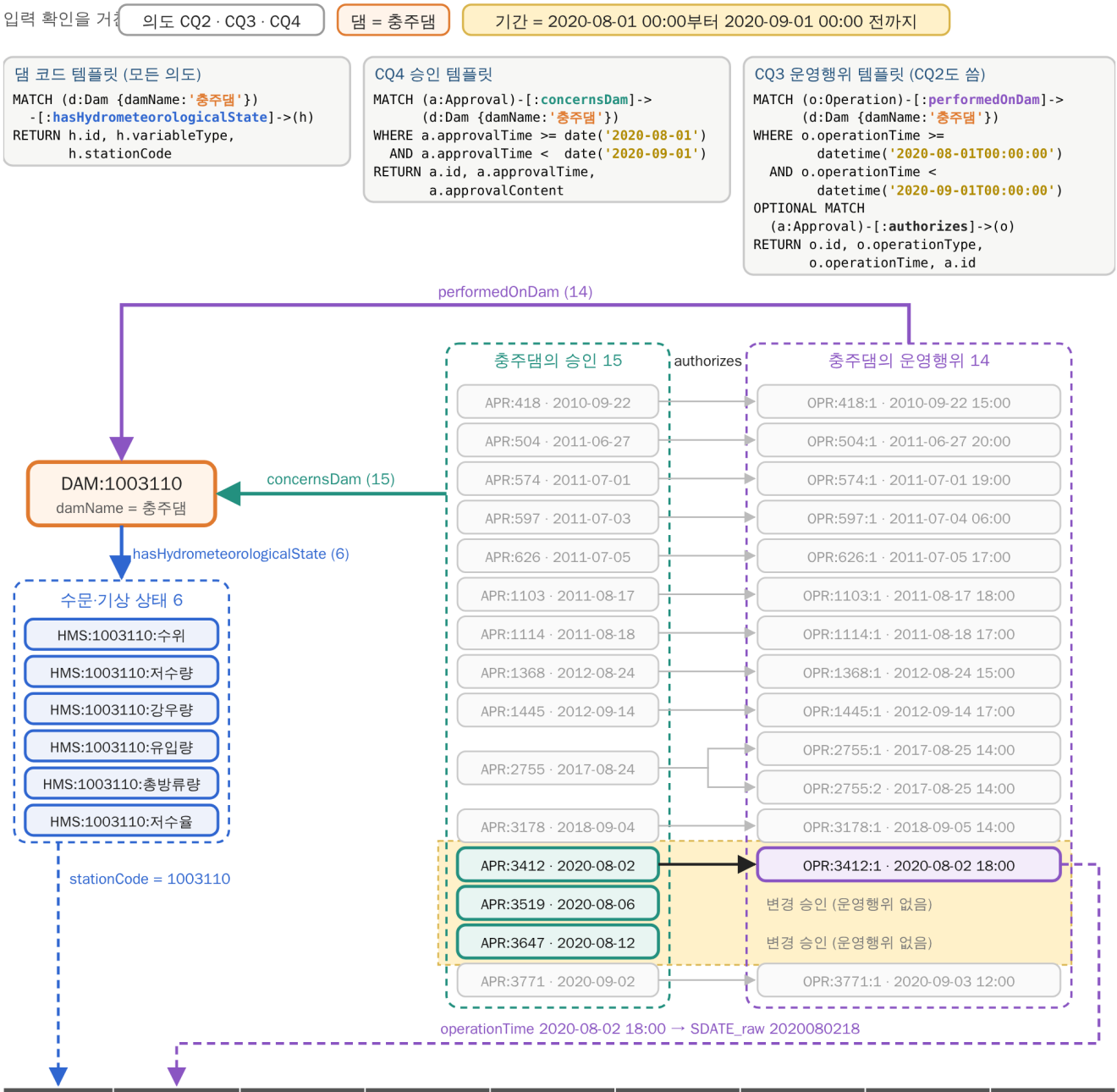
과정 2. 입력 확인 (코드)

- 의도가 CQ1~CQ6 안인지, 댐이 4 개 댐 안인지 확인하고 이름을 표준 댐명으로 바꾼다("충주" → 충주댐).
- 기간의 끝 날짜는 그 날 전체로 읽어 [2020-08-01 00:00, 2020-09-01 00:00)으로 바꾼다. 이 기간과 표준 댐명을 템플릿에 넣는다. 확인을 통과하지 못하면 조회하지 않고 "질의 불가: 이유"를 답변 과정으로 넘긴다.

과정 3. KG 검색과 외부 저장소 조회 (코드)

- 원리: 고른 의도마다 미리 정한 Cypher 템플릿에 댐 이름과 기간만 바꿔 넣어 실행한다. 템플릿은 KG 의 Relation 을 따라가고, 측정값과 원문은 KG 의 Data Property 값(키)으로 외부 저장소에서 가져온다.
- 그림 13 의 맨 위는 입력 확인을 거친 값과 이 질문에 쓰는 템플릿 세 개이다. 템플릿에서 주황 글자(충주댐)와 노랑 글자(날짜)가 그 값이 들어간 자리이고, 파랑·초록·보라 글자는 아래 KG 에서 같은 색의 Relation 이다.
- 댐 코드 템플릿은 DAM:1003110 에서 hasHydrometeorologicalState 를 따라 HMS 6 개를 찾고 stationCode 1003110 을 얻는다. 측정 저장소를 찾아갈 키이다.
- CQ4 승인 템플릿은 concernsDam 으로 충주댐에 이어진 승인 15 개 가운데 승인일이 2020-08-01 이상 2020-09-01 미만인 것을 고른다 → APR:3412, APR:3519, APR:3647. 뒤의 둘은 변경 승인이라 운영행위가 없다.
- CQ3 운영행위 템플릿은 performedOnDam 으로 충주댐에 이어진 운영행위 14 개 가운데 시각이 기간 안인 것을 고르고, authorizes 를 거꾸로 따라 그것을 허가한 승인을 찾는다 → OPR:3412:1 ← APR:3412. 노란 띠 밖의 흐린 Entity 는 기간 조건에서 걸러진 것이다.

- CQ2 는 CQ3 과 함께 있으므로 운영행위의 시각마다 그 시각의 상태를 읽는다. 맨 아래 점선처럼 HMS 의 stationCode(1003110)를 DAM_CD 열에서, OPR:3412:1 의 operationTime(2020-08-02 18:00)을 SDATE_raw 열(2020080218)에서 찾으면, 그 행이 그 시각의 측정값이다. 정각이 아닌 시각은 앞뒤 정시 두 행을 함께 읽는다.
- 다른 의도가 있는 질문은 표 3 의 경로를 따른다.



DAM_CD	SDATE_raw	DATA1_수위 (EL.m)	DATA2_저수량 (MCM)	DATA3_강우량 (mm)	DATA4_유입량 (CMS)	DATA5_source_label_unknown	DATA6_총방류량 (CMS)	DATA7_저수율 (%)
1003110	2020080217	139.37	2111.215	0.711	4693.654	4693.654	707.265	76.8
1003110	**2020080218**	**139.55**	**2126.472**	**1.901**	**4945.201**	**4945.201**	**707.145**	**77.3**
1003110	2020080219	139.69	2138.381	0.273	4015.509	4015.509	707.453	77.8

측정 저장소 kwater_mywater_hydr_H_aligned_wide.csv.gz의 행 (앞의 source_file 등 3열은 생략). 고른 행의 값이 [MEA:1003110:2020080218]이 된다.

실선은 Neo4j에 저장된 Relation, 템플릿의 주황·노랑 글자는 입력 확인을 거친 댐 이름과 기간이 들어간 자리이다. 노란 띠는 기간 안의 Entity이다. 진하게 칠한 Entity가 이 질문에서 찾은 것이고 흐린 것은 기간 밖이라 걸러진 것이다. 점선은 KG의 값으로 측정 저장소의 행을 찾아가는 길이다.

그림 13. KG 검색과 외부 저장소 조회 — 템플릿이 따라가는 Relation 과 기간 조건, KG 의 값으로 찾는 측정 저장소의 행

표 3. 의도별 Cypher 템플릿이 따라가는 Relation 과 외부 저장소

의도	따라가는 Relation (Cypher 템플릿)	외부 저장소
CQ1 현재 상태	Dam → hasHydrometeorologicalState → HMS (stationCode)	측정: 저장된 가장 최근 시각의 행
CQ2 과거 상태	(CQ3 과 함께면) 운영행위의 operationTime	측정: 그 시각의 행, CQ3 이 없으면 기간 안의 행
CQ3 운영행위	Operation → performedOnDam → Dam, Approval → authorizes → Operation	—
CQ4 승인	Approval → concernsDam → Dam	—
CQ5 운영기준	Criterion → appliesToDam → Dam, Criterion → supportedBy → EvidenceSource	문서: sourceLocator 의 원문
CQ6 근거	조회한 Entity → supportedBy → EvidenceSource	—

과정 4. 근거 묶음 (코드)

- 조회 결과를 의도별 구역으로 나누고, 항목마다 출처 번호를 붙인다. 출처 번호는 KG 의 Entity ID 이고, 측정값은 [MEA:댐 코드:시각]이다.
- 값은 저장소의 원값 그대로 쓰고 계산한 값은 넣지 않는다. 끝에 근거 사용 원칙 3 문장을 붙인다.

```
[질문] 2020년 8월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘.
[대상] 충주댐, 기간 2020-08-01 ~ 2020-08-31

[근거 1. 운영행위와 허가한 승인]  (CQ3) 운영행위의 시각은 비고에 적힌 시각, 없으면 승인된 방류 시작 시각이다.
[OPR:3412:1] 방류, 시각 2020-08-02 18:00 ← 승인 [APR:3412]

[근거 2. 운영 시각의 상태]  (CQ2)
[MEA:1003110:2020080218] 2020-08-02 18:00  수위 139.55 EL.m, 저수량 2126.472 MCM, 강우량 1.901 mm,
   유입량 4945.201 CMS, 총방류량 707.145 CMS, 저수율 77.3 %

[근거 3. 방류 승인]  (CQ4)
[APR:3412] 승인일 2020-08-02: 방류시작 2020-08-02 18:00; 접수방류량 3000㎥/s
[APR:3519] 승인일 2020-08-06: 방류시작 2020-08-02 18:00; 접수방류량 7000㎥/s; 비고: 방류기간 (당초) … (변경)
…
[APR:3647] 승인일 2020-08-12: 방류시작 2020-08-06 06:00; 접수방류량 7000㎥/s; 비고: 수문방류 변경승인(방류기
간연장) …

[근거 사용 원칙] 근거에 있는 내용만 사실로 쓰고 출처 번호를 붙인다 / 비교·해석은 판단임을 밝힌다 / 없으면 "자료에 없
음"
```

의도별 구역 출처 번호 = KG의 Entity ID, 측정값은 MEA 원칙 3문장

그림 14. 근거 묶음 — LLM 에 주는 근거 전체

과정 5. 답변 (LLM)

- LLM 에 답변 지시문과 근거 묶음을 준다. 지시문: 근거 묶음에 있는 내용만 사실로 쓰고 출처 번호를 붙인다, 비교·해석은 "판단"으로 나누고 쓴 근거를 적는다, 없으면 "자료에 없음".
- 주의 사항도 함께 준다: 운영행위 시각은 비고의 시각이거나 승인된 방류 시작 시각이다, "현재"는 저장된 가장 최근 시각이며 실시간이 아니다, 방류 사유는 자료에 없으므로 단정하지 않는다.

4.3 결과: 답변

- 그림 15 가 이 질문의 답변이다. "근거"에는 승인 세 건의 방류량과 기간이 [APR:3412], [APR:3519], [APR:3647] 과 함께, 방류 시작 시각의 수문 상태가 [MEA:1003110:2020080218]과 함께 쓰여 있다.
- "판단"에는 방류량이 3000㎥/s 에서 7000㎥/s 로 늘고 기간이 연장되었다는 비교가 쓴 근거와 함께 있다. 빨간 글씨 문장은 자료에 없는 사유를 추정한 것으로, 지시문의 "사유를 단정하지 않는다"를 어겼다.
- 근거 묶음의 운영행위 [OPR:3412:1]은 답변에 출처 번호로 쓰이지 않았다.

```
근거
- 2020년 8월 2일 18:00에 방류가 승인되었으며, 방류량은 3000㎥/s로 접수되었습니다. [APR:3412]
- 이후 8월 6일에 방류 승인이 변경되어 방류량이 7000㎥/s로 증가하고, 방류 기간이 8월 2일 18:00부터 8월 16일 24:00까지로
  확정되었습니다. [APR:3519]
- 8월 12일에는 방류 기간이 다시 연장되어 8월 6일 06:00부터 8월 22일 00:00까지로 확정되었습니다. [APR:3647]
- 방류가 시작된 2020년 8월 2일 18:00 기준, 수위는 139.55 EL.m, … 저수율은 77.3%입니다.
[MEA:1003110:2020080218]

판단
- 방류량은 처음 3000㎥/s에서 이후 7000㎥/s로 증가했습니다. 방류 기간은 여러 차례 변경되며 최종적으로 8월 22일 00:00까지
  연장되었습니다. (근거: [APR:3412], [APR:3519], [APR:3647])
- … 이는 댐의 수문 상황과 유입량 변화에 따라 조정된 것으로 판단됩니다. (근거: [APR:3412], …)
```

사실마다 출처 번호 판단마다 쓴 근거 빨간 글씨: 자료에 없는 사유를 추정한 문장 (지시문의 "사유를 단정하지 않는다"를 어김)

그림 15. 답변 — 근거(출처 번호)와 판단(쓴 근거)으로 나뉜다

5. 검토할 점과 다음 계획

1. 답할 수 있는 질문의 범위 (온톨로지, 검색·답변)

- 온톨로지는 CQ1~CQ6 에 답할 수 있는 구조로 만들었고, 검색은 CQ 마다 정한 Cypher 템플릿으로 한다(4 장). 그래서 지금 구조는 질문의 의도가 CQ1~CQ6 에 해당할 때만 답할 수 있다.
- 의도 목록에 없는 질문은 과정 1 에서 intent 가 비어 조회하지 않는다. 예: 방류 사유를 묻는 질문, 4 개 댐 밖의 댐이나 두 댐을 함께 묻는 질문.

2. 과제 발표와 시연

- 11/20 과제 발표와 11/26 시연은 지금 구조(4 장)로 한다.

3. 연구

- 온톨로지 생성 방식을 다시 검토한다.
- 검색·답변과 검증에 대한 선행 논문을 더 찾아 읽고 지금 방법과 비교한 뒤, 더 좋은 결과를 내는 방법을 적용한다. 검증은 11/02 회의에 따라 다른 연구들이 KG·온톨로지를 어떻게 검증했는지부터 조사한다.

부록 A. 용어 정리

표 4. 용어 정의

용어	본 연구에서의 정의
Relation 잇기	온톨로지의 Relation 정의에 맞는 Class 사이의 조합 가운데, 키로 실제로 이어질 Entity 쌍(시작 Entity → 끝 Entity)을 찾아 잇는 일.
키	두 자료나 두 Entity 가 같은 대상에 관한 것임을 알려 주는 값. 예: 승인 순차번호 504, 댐 코드 1003110. Relation 을 이을 때와 외부 저장소에서 행을 찾을 때 같은 뜻으로 쓴다.
시작·끝 Entity	Relation 한 줄의 출발점과 도착점. "APR:504 —concernsDam→ DAM:1003110"에서 APR:504 가 시작 Entity, DAM:1003110 이 끝 Entity.
개체 해소	자료마다 다르게 적힌 같은 대상(충주, 충 주 댐, 1003110)을 하나의 기준 식별자에 대응시키는 일. 문헌의 entity disambiguation.
기준 식별자	같은 대상을 가리키는 표기들을 모으는 기준 ID. 본 연구는 K-water 댐 코드, GraphAide 는 Wikidata 의 QID.
Neo4j	Entity 와 Relation 을 그래프(점과 선) 모양으로 저장하는 데이터베이스.
Cypher	Neo4j 에 Entity 와 Relation 을 넣고 찾을 때 쓰는 명령 문장. MERGE(없으면 만듦), MATCH(찾음), SET(값을 넣음) 등.
지시문	LLM 에 질문과 함께 주는 지침 글(프롬프트). 의도·값 추출용과 답변용이 따로 있다.
조회 템플릿	CQ 마다 미리 정해 둔 Cypher 문. 댐 이름과 기간만 바꿔 넣어 실행한다.

용어	본 연구에서의 정의
의도	질문이 요구하는 CQ(CQ1~CQ6). 한 질문에 여러 개가 있을 수 있다.
근거 묶음	KG 와 저장소에서 찾은 근거를 의도별 구역과 출처 번호를 붙여 LLM 에 주는 글.
출처 번호	근거 항목마다 붙이는 표시. KG 의 Entity ID 이거나 측정값의 [MEA:댐 코드:시각].
환각	LLM 이 자료에 없는 사실을 지어내는 것.

부록 B. Ontology 가 답해야 할 질문과 관계

표 5. 세부 질문 (CQ1~CQ6)

번호	질문 대상	세부 질문
CQ1	현재 수문·기상 상태	대상 댐의 현재 또는 기준시점의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 무엇인가?
CQ2	과거 수문·기상 상태	선택된 과거 시점 또는 사례에서 대상 댐의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 어떠했는가?
CQ3	과거 운영행위	해당 과거 시점 또는 사례에서 실제로 수행된 댐 운영행위는 무엇이었는가?
CQ4	과거 방류 승인	해당 과거 시점 또는 사례에서 어떤 방류 승인이 있었으며, 그 승인내용은 무엇인가?
CQ5	현재 운영기준	현재 대상 댐에 적용되는 운영기준은 무엇이며, 현재 운영 판단과 관련된 기준값은 무엇인가?
CQ6	공식 근거	각 수문·기상 상태, 운영행위, 승인내용 및 운영기준을 확인할 수 있는 공식 자료 또는 근거문서는 무엇인가?

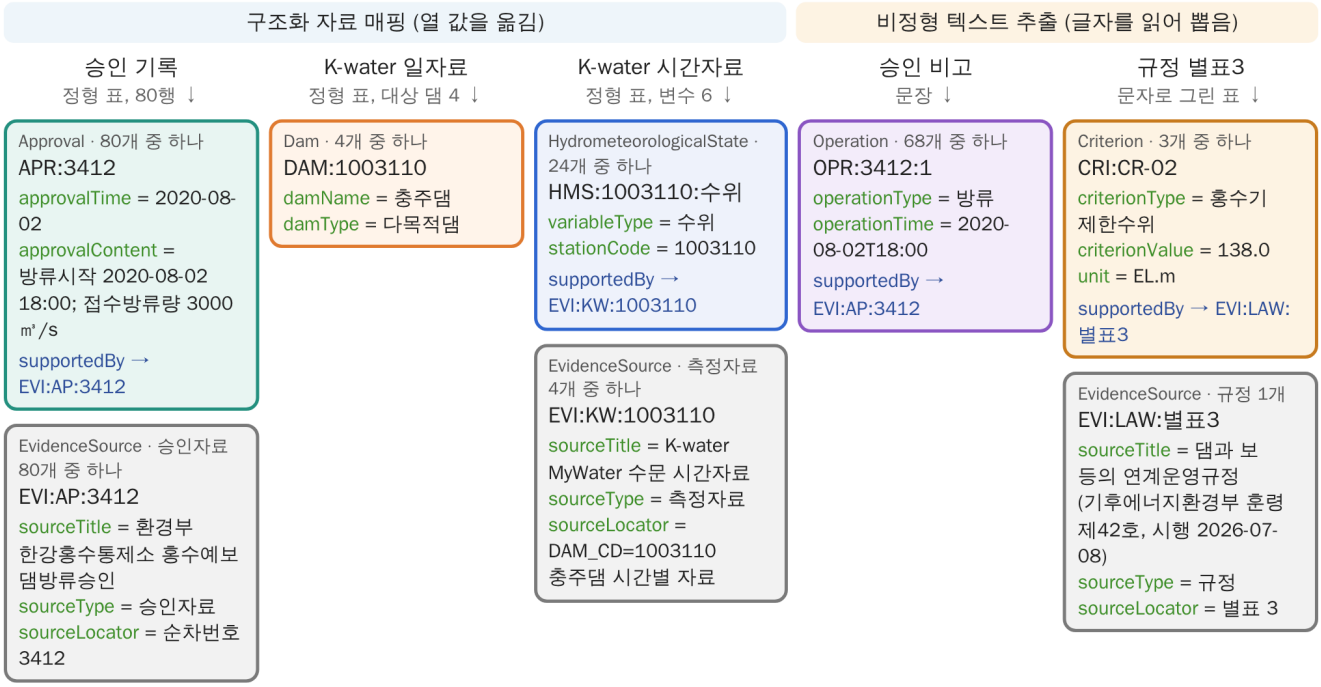
표 6. Relation (9 개)

Subject	Relation	Object	의미
Dam	hasHydrometeorologicalState	HydrometeorologicalState	댐이 해당 수문·기상 변수 정보와 연결됨
Operation	performedOnDam	Dam	운영행위가 특정 Dam 에서 수행됨
Approval	authorizes	Operation	Approval 이 특정 Operation 을 승인함
Approval	concernsDam	Dam	Approval 이 특정 Dam 에 관한 것임
Criterion	appliesToDam	Dam	운영기준이 특정 Dam 에 적용됨
HydrometeorologicalState	supportedBy	EvidenceSource	상태를 확인할 수 있는 근거자료
Operation	supportedBy	EvidenceSource	운영행위를 확인할 수 있는 근거자료
Approval	supportedBy	EvidenceSource	승인을 확인할 수 있는 근거자료
Criterion	supportedBy	EvidenceSource	운영기준을 확인할 수 있는 근거자료

부록 C. 10/30 자료까지의 결과: 자료 준비, Class·Data Property 채우기

- 자료 준비: 승인 기록 3,929 행에서 중복 7 행을 지우고 대상 4 개 댐의 80 행을 남겼다. K-water 시간자료 두 파일을 이어 붙여 4 개 댐의 508,246 행을 측정 저장소에, 규정 XML 에서 꺼낸 별표 3 원문을 문서 저장소에 두었다.
- Class·Data Property 채우기: 정형 표는 구조화 자료 매핑으로, 승인 비고와 별표 3 은 비정형 텍스트 추출로 Entity 와 속성값을 만들었다. 그림 16 은 충주댐에 관한 실제 Entity 를 자료마다 하나씩 보인 것이다. 결과는 kgv4_entities.csv(Entity 264 개)와 kgv4_properties.csv(속성값 616 개)이다.

- 이 단계에서 이어진 Relation 은 같은 행·같은 자료에서 함께 만든 근거(supportedBy) 175 개뿐이었다. 나머지 Relation 243 개를 2 장에서 이었다.



충주댐에 관한 실제 Entity를 자료마다 하나씩 보인 것이다. 초록 글자가 Data Property, 굵은 글자가 그 값이다. 같은 Class의 다른 Entity도 같은 Data Property를 가진다. 이 단계에서 이어진 Relation은 같은 행·같은 자료에서 함께 만든 근거로 가는 supportedBy뿐이다.

그림 16. 10/30 자료까지 만든 Entity — 자료마다 충주댐의 실제 Entity 하나와 그 Data Property 값

부록 D. 근거 문헌의 원리

D.1 Purohit 외(2024) — Relation 잇기(개체 해소), 검색·답변

- 무엇인가: 여러 출처의 정형·비정형 자료로 KG 를 만들고, 그 KG 를 조회·추론하는 질의 시스템 GraphAide 를 제안한 학회 논문이다(IEEE BigData 2024). OpenAI LLM, Chroma 벡터 DB, Neo4j, LangChain 으로 구현했다.
- KG 생성 단계의 개체 해소: 기준 지식베이스로 DARPA Wikidata(DWD)를 쓰고, QID 마다 설명 문서를 만들어 벡터 DB 에 넣었다. 온톨로지와 DWD 에 기반한 에이전트가 글에서 대상의 표기를 뽑고, RAG 기반 후보 재순위로 가장 가까운 QID 를 고른다. → 기준 식별자에 대응시키는 원리를 적용했다. 대상이 댐 4 개라 이번에는 코드 일치와 대응 표로 했고, 벡터 DB 와 LLM 재순위는 대상과 자료가 늘어날 때 적용을 검토한다.
- 질의 단계: LLM 이 질문을 여러 형태로 확장하고, 이를 부분 그래프 조회 템플릿으로 바꿔 Cypher 를 자동 생성한다. 그래프 조회 결과와 벡터 검색 결과를 합쳐 LLM 의 맥락으로 주고, 답변을 모아 하나의 응답을 만든다. 정해진 순서(chain)와 에이전트를 함께 쓴다. → 이번 시험에는 KG 조회 결과를 LLM 의 근거로 주는 원리를 적용하고 Cypher 는 CQ 마다 미리 정한 템플릿을 썼다. 질의 확장, Cypher 자동 생성, 벡터 검색, 에이전트는 답할 수 있는 질문의 범위를 넓힐 때 적용을 검토한다.
- 평가: RAGAS 지표와 전문가 정성 평가를 쓴다고 밝혔으며, 정식 평가는 향후 과제로 두었다.

D.2 Huang 외(2026) — 저장

- 추출한 지식을 JSON 으로 통합하고, DDO 의 의미·계층 관계에 따라 Python 스크립트가 JSON 키-값을 읽어 Cypher 문을 만든다. 이 문들로 DDKG 의 Entity 를 일괄 생성하고 Entity 사이의 관계를 만들어 Neo4j 에 적재한다.
- 저장소로 Neo4j 를 고른 이유: Cypher 로 조회·추가·삭제·수정이 효율적이고, 다중 홉 추론과 복잡한 질의에 강하다.
- 구축한 DDKG 는 Entity 201,362 개, Relation 953,447 개이다. → JSON 대신 CSV 를 모은 형태로 쓴 것 외에는 같은 순서(모음 → 스크립트로 Cypher 생성 → 일괄 적재)를 따랐다.

D.3 Zhou 외(2023) — KG 밖에 두는 저장소

- 무엇인가: 댐 안전 모니터링을 위한 BIM·온톨로지 기반 지식관리 저널 논문이다(Automation in Construction). OntoDSMS 는 이 논문이 만든 댐 안전 모니터링 시스템(DSMS) 도메인 온톨로지의 이름이고, D2RQ 는 이 논문이 쓴 기존 오픈소스 도구이다.

- 데이터를 원래 저장소에 둠: D2RQ 는 관계형 DB 에 접근하는 가상 RDF 그래프를 제공하며, RDB 의 내용을 RDF 저장소에 복제하지 않고 RDF 로 접근하게 한다(원문 6 쪽).
- 키로 대응: 매핑 파일에서 표의 ID 열 값으로 인스턴스 식별자를 만들고(uriPattern), 표는 Class, 열은 DatatypeProperty, 행은 인스턴스, 셀은 값에 대응시킨다(원문 6 쪽 그림 3, 7 쪽).
- 질의할 때 읽어 옴: D2RQ 는 RDF 질의를 SQL 문으로 번역해 RDB 에서 실행한다(원문 7 쪽). 이력 데이터와 실시간 수집 데이터를 MySQL 에 둔 채 온톨로지와 대응시켰고(원문 10 쪽), 질의 1 에서 4 번 구간 수직 변위 센서의 2015-2-28 이후 모니터링 데이터를 조회했다.
- → 데이터를 원래 저장소에 두고 키로 KG 와 이어 질의할 때 읽어 오는 원리를 따랐다. 매핑 파일과 SQL 번역 대신, KG 에서 키를 얻은 뒤 코드가 CSV 에서 행을 찾는다.

부록 E. 참고문헌

- Huang, J., Hou, W., Zhang, S., Du, Z., & Yin, J. (2026). Dam defects ontology and knowledge graph construction from multi-source hazard data using large language models. Advanced Engineering Informatics, 76, 104987.
- Purohit, S., Chin, G., Mackey, P. S., & Cottam, J. A. (2024). GraphAide: Advanced Graph-Assisted Query and Reasoning System. 2024 IEEE International Conference on Big Data. https://doi.org/10.1109/BigData62323.2024.10825705
- Zhou, Y., Bao, T., Shu, X., Li, Y., & Li, Y. (2023). BIM and ontology-based knowledge management for dam safety monitoring. Automation in Construction, 145, 104649.